DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 7

Front-end developer

Profession 7 of 10 in Development and IT.

What the profession means

A front-end developer builds the part of a website or app that users see and use: layout, interaction, accessibility and performance. The work happens in HTML, CSS and JavaScript, often with a framework such as React.

What professionals do every day

Turns designs into components, makes them accessible and responsive, connects screens to data from the back end, tests in several browsers and fixes performance problems.

Why businesses hire them, and what problems they solve

Users decide in seconds whether a product is trustworthy. Front-end developers make it fast, usable and inclusive, which also helps conversions and search rankings.

- Pages that do not work with a keyboard or screen reader.
- Slow sites with heavy images and scripts.
- Layouts that break on some phones or browsers.

Skills you need

- HTML semantics and forms.
- CSS layout, responsive design and animation basics.
- JavaScript for interaction, the DOM and events.
- Accessibility (WCAG basics), performance and browser testing.
- A component framework such as React (see Profession 9).

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Visual Studio Code	Code editor	Free	Free	code.visualstudio.com
Chrome DevTools	Debug layout, network and performance	Free	Free	developer.chrome.com/docs/devtools
web.dev	Official guidance on performance and accessibility (Google)	Free	Free	web.dev
axe DevTools	Accessibility checks in the browser	Free extension	Paid tiers	www.deque.com/axe/
Lighthouse	Performance and accessibility audits	Free	Free	developer.chrome.com/docs/lighthouse

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	HTML and CSS layouts that adapt; forms with clear errors.
Intermediate	Interactive components (accordions, menus, tabs) that work with the keyboard.
Advanced	Performance budgets, lazy loading, component libraries and automated accessibility tests.

Projects by level

- **Beginner:** A responsive landing page with a working FAQ.
- **Intermediate:** A product page with a gallery, filters and a cart icon.
- **Advanced:** A component library with documented accessible states.

Client-ready portfolio project

A responsive, accessible site for a real or fictional business, with a Lighthouse report over 90 in performance and accessibility on mobile (check the result on your own run), and a short accessibility statement.

Worked example

Worked example. Making a menu accessible: use `<nav>`, make each link keyboard reachable, and give the open state to assistive technology with `aria-expanded`.

Assignment with full solution

ASSIGNMENT

An accessible FAQ accordion

Business problem: A website's FAQ must open and close answers with a mouse, a touch screen and a keyboard, and tell screen readers which answer is open.

Steps

1. Use a real `<button>` for each question, not a styled div.
2. Link each button to its answer with `aria-controls`, and set `aria-expanded` to true or false.
3. Toggle the `hidden` attribute on the answer when the button is pressed.
4. Check that Tab and Enter (or Space) open and close each answer.

Expected output: Each question opens and closes its answer; the screen reader announces expanded or collapsed; the keyboard works.

SOLUTION (ANSWER KEY)

Why it works. A native button gets keyboard focus and Enter/Space handling for free. `aria-expanded` tells assistive technology the state, and `aria-controls` links the button to its panel. Removing the `hidden` attribute shows the answer. Tested in a browser: clicking each question toggles its panel and the state attribute.

HTML

```
<!-- faq.html : accessible accordion (buttons, aria, keyboard) -->
<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1"><title>FAQ</title>
<style>
  body { font-family: system-ui, sans-serif; margin: 16px; color: #1d1640; }
  .q { width: 100%; text-align: left; padding: 12px; font-size: 1rem; border: 1px solid #ccc;
background: #fff; }
  .a { padding: 12px; border: 1px solid #ccc; border-top: 0; }
  .a[hidden] { display: none; }
</style></head>
<body>
<h1>Delivery questions</h1>
<h3><button class="q" aria-expanded="false" aria-controls="a1" id="b1">Do you deliver on
Sundays?</button></h3>
<div class="a" id="a1" role="region" aria-labelledby="b1" hidden><p>Yes, from 8 am to 6 pm.
</p></div>
<h3><button class="q" aria-expanded="false" aria-controls="a2" id="b2">What is the minimum
order?</button></h3>
<div class="a" id="a2" role="region" aria-labelledby="b2" hidden><p>Rs 300 across all items.
</p></div>
<script>
  document.querySelectorAll('.q').forEach((btn) => {
    btn.addEventListener('click', () => {
      const panel = document.getElementById(btn.getAttribute('aria-controls'));
      const open = btn.getAttribute('aria-expanded') === 'true';
      btn.setAttribute('aria-expanded', String(!open));
      panel.hidden = open;
    });
  });
</script>
</body></html>
```

Common mistakes

- Using clickable divs instead of buttons and links.
- Removing the focus outline without a visible replacement.
- Shipping images without width and height, which causes layout shift.
- Testing only with a mouse.

Freelancing opportunities

- Landing page and interface work for product teams and agencies.
- Accessibility audits and fixes for existing sites.
- Freelance briefs for 'HTML/CSS', 'JavaScript', 'React front-end', 'accessibility fix'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- web.dev Learn
- MDN: Learn web development
- WCAG 2.2 quick reference (W3C)

Part of **Master Freelancing: Development and IT**. Back to the chapter list on skillnavora.in · Open the full textbook. Official links in this chapter were not checked live; confirm before relying on them.